Topic -4: Dynamic Programming

AIMLCZG512-Deep Reinforcement Learning

Lecture Notes

1. Topics Covered

1. Dynamic programming as a planning method for finite MDPs with known dynamics.
2. Bellman equations and the recursive structure of value functions.
3. Policy evaluation, policy improvement, and the policy improvement theorem.
4. Policy iteration and value iteration as dynamic programming algorithms.
5. Race-car and grid examples for understanding Bellman backups.
6. In-place and not-in-place updates, asynchronous DP, and generalized policy iteration.
7. Practical issues, key terms, and review questions involving conceptual, numerical, and modelling exercises.

2. Dynamic Programming for Reinforcement Learning

Dynamic programming (DP) refers to a family of algorithms for solving problems by using recursive relationships among subproblems. In reinforcement learning, DP methods use the Bellman equations to compute value functions and policies when a complete model of the finite MDP is available.

DP is important in this course for two reasons. First, it gives the cleanest mathematical solution to finite MDPs. Second, many later reinforcement learning algorithms can be understood as sample-based or approximate versions of DP backups.

2.1. Recursive Structure of Value Functions

For a policy π, the state-value function is the expected return from state s when actions are selected according to π:

$$v_\pi(s) \doteq \mathbb{E}_\pi \left[G_t \mid S_t = s \right]. \tag{1}$$

The Bellman equation expresses this value recursively:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s') \right]. \tag{2}$$

The value of the present state is written in terms of immediate reward plus the discounted value of the next state. **This recursive decomposition is the basis of DP: Bellman equations are turned into backups that update one state using immediate rewards and the estimated values of successor states**.

For optimal control, the Bellman optimality equation is

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]. \tag{3}$$

It says that the optimal value of a state is obtained by choosing the action whose one-step lookahead gives the largest expected return.

2.2. Why DP is Useful for RL

Reason	Meaning for RL
Bellman recursion	Values are computed using immediate rewards and values of successor states.
Policy improvement	Once v_π is known, a better policy can be formed by acting greedily with respect to this value function.
Planning from a model	If $p(s', r \mid s, a)$ is known, one can improve the policy without waiting for real interaction samples.
Foundation for later methods	Monte Carlo, TD learning, SARSA, Q-learning, and DQN can be seen as replacing exact expected backups by sampled or approximate backups.

The limitation is equally important: classical DP requires a complete model and finite state-action spaces. This makes it theoretically central but not directly scalable to many real-life RL problems.

3. Policy Evaluation

Policy evaluation is the problem of computing the value function v_π for a fixed policy π. It answers the prediction question: *if the agent follows this policy, how good is each state?*

Starting from the Bellman equation for v_π, iterative policy evaluation uses the update

$$v_{k+1}(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_k(s') \right], \qquad s \in \mathcal{S}. \tag{4}$$

Here v_k is the current approximation and v_{k+1} is the improved approximation after one sweep. Each sweep applies the update to every state.

Iterative Policy Evaluation

Input: policy π to be evaluated.
Parameter: small threshold $\theta > 0$.
Initialize: $V(s)$ arbitrarily for all $s \in \mathcal{S}^+$, with $V(\text{terminal}) = 0$.

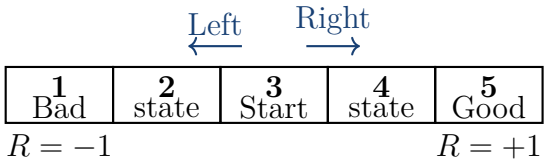
Loop:
  $\Delta \leftarrow 0$
  For each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$ *store the old value*
    $V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s') \right]$
    apply the Bellman expectation backup
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ *record the largest change*
Until $\Delta < \theta$.

Output: an approximation $V \approx v_\pi$.

Working of the algorithm. The policy is fixed. The algorithm repeatedly replaces the old value of each state by the expected one-step return under that same policy. If the largest change in a sweep is small, the value function has nearly stabilized.

3.1. One-Dimensional Grid Example

Consider a five-cell line with states $1, 2, 3, 4, 5$. State 1 is the bad terminal, state 5 is the good terminal, and state 3 is the start state. The nonterminal states are $2, 3, 4$. Actions move left or right deterministically. Entering the bad terminal gives reward -1; entering the good terminal gives reward $+1$; all other transitions give reward 0. Let $\gamma = 1$ and initialize all nonterminal values to zero.



Three policies are evaluated: mostly left, equal probability, and mostly right. In each case, the values are shown for the three nonterminal states $(2, 3, 4)$ after four sweeps.

Policy A: mostly left. Let $\pi(L) = 0.8$ and $\pi(R) = 0.2$. Starting from zero values, the four sweeps give

Sweep	$(V(2), V(3), V(4))$
1	$(-0.80,\ 0.00,\ 0.20)$
2	$(-0.80,\ -0.60,\ 0.20)$
3	$(-0.92,\ -0.60,\ -0.28)$
4	$(-0.92,\ -0.792,\ -0.28)$

The starting cell becomes negative because the policy more often moves toward the bad terminal.

Policy B: equal probability. Let $\pi(L) = 0.5$ and $\pi(R) = 0.5$. The first sweep gives

$$(V(2), V(3), V(4)) = (-0.50,\ 0.00,\ 0.50).$$

Later sweeps keep the middle value balanced because the left and right directions are equally likely and the terminal rewards are symmetric.

Policy C: mostly right. Let $\pi(L) = 0.2$ and $\pi(R) = 0.8$. Starting from zero values, the four sweeps give

Sweep	$(V(2), V(3), V(4))$
1	$(-0.20,\ 0.00,\ 0.80)$
2	$(-0.20,\ 0.60,\ 0.80)$
3	$(0.28,\ 0.60,\ 0.92)$
4	$(0.28,\ 0.792,\ 0.92)$

The starting cell becomes positive because the policy more often moves toward the good terminal.

Interpretation. Policy evaluation does not change the policy. It only computes how good that fixed policy is. A better policy gives larger values to states from which the good terminal is more likely to be reached.

4. Policy Improvement

Policy improvement uses a value function to obtain a better policy. After computing v_π, we can look one step ahead from each state and choose an action that gives the highest expected return according to v_π:

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma v_\pi(s')\right]. \tag{5}$$

A greedy improved policy is

$$\pi'(s) = \arg\max_a q_\pi(s,a) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma v_\pi(s')\right]. \tag{6}$$

Policy Improvement Theorem (Optional Topic)

Statement. Suppose two deterministic policies π and π' satisfy

$$q_\pi(s, \pi'(s)) \geq v_\pi(s), \qquad \forall s \in \mathcal{S}. \tag{7}$$

Then π' is at least as good as π:

$$v_{\pi'}(s) \geq v_\pi(s), \qquad \forall s \in \mathcal{S}. \tag{8}$$

Purpose. The theorem justifies policy improvement. Once v_π is known, we can choose at each state an action that is greedy with respect to v_π. If the greedy choice is no worse than the old policy action in every state, the resulting policy cannot be worse.

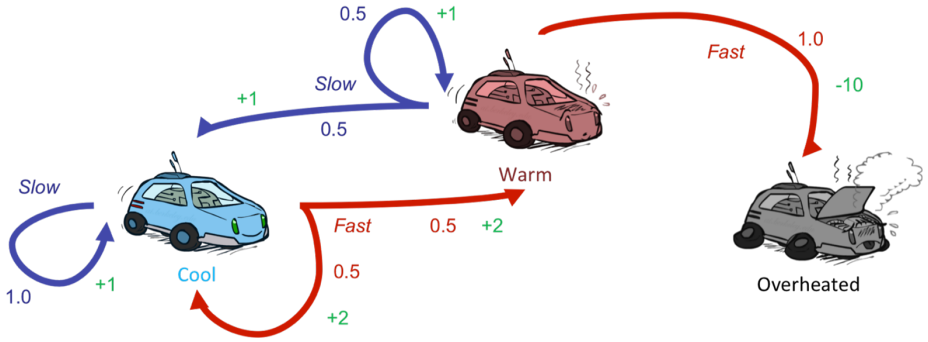
Proof idea. Starting from one state s, take the improved action once and then use the same comparison repeatedly:

$$\begin{aligned} v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s\right] \\ &\leq \cdots \leq v_{\pi'}(s). \end{aligned}$$

Thus, repeatedly following the improved choices gives an expected return at least as large as following the old policy. If the greedy policy no longer changes, it satisfies the Bellman optimality equation and is optimal.

5. Race-Car Example

The race-car example is a compact MDP for understanding policy and value updates. The car can be in three states: *Cool*, *Warm*, and *Overheated*. The actions are *Slow* and *Fast*. Going fast can give higher immediate reward, but it may increase the risk of overheating.



Race-car MDP used for demonstrating DP updates. Blue transitions correspond to slow action; red transitions correspond to fast action.

State	Action	Transition and reward interpretation
Cool	Slow	Remain cool with probability 1.0 and reward +1.
Cool	Fast	Move to Cool with probability 0.5 and to Warm with probability 0.5; reward +2.
Warm	Slow	Move to Cool with probability 0.5 and remain Warm with probability 0.5; reward +1.
Warm	Fast	Move to Overheated with probability 1.0 and reward −10.
Overheated	–	Terminal or absorbing failure state for this demonstration.

The best action is not decided only by immediate reward. The fast action in Warm has a large negative consequence, so a good policy avoids it even if going fast is attractive elsewhere.

6. Policy Iteration (Optional Topic)

Policy iteration alternates between policy evaluation and policy improvement. Evaluation estimates the value function for the current policy. Improvement makes the policy greedy with respect to that value function.

Policy Iteration (using iterative policy evaluation)

Goal: estimate $\pi \approx \pi_*$ and $V \approx v_*$.

1. Initialization

Choose $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$.

2. Policy Evaluation

Repeat until the current policy value is sufficiently accurate:

1. Set $\Delta \leftarrow 0$.
2. For each $s \in \mathcal{S}$:

$$v \leftarrow V(s),$$
$$V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) \left[r + \gamma V(s')\right],$$
$$\Delta \leftarrow \max(\Delta, |v - V(s)|).$$

3. Stop evaluation when $\Delta < \theta$.

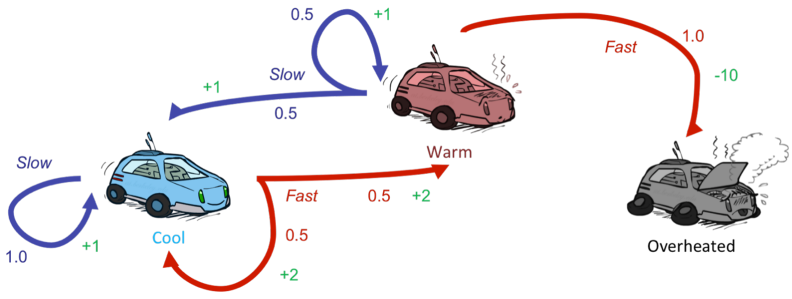
3. Policy Improvement

Set *policy-stable* $\leftarrow$ true. For each $s \in \mathcal{S}$:

$$\textit{old-action} \leftarrow \pi(s),$$
$$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s')\right].$$

If *old-action* $\neq \pi(s)$, set *policy-stable* $\leftarrow$ false. If *policy-stable* is true, stop and return V and π; otherwise go back to policy evaluation.

6.1. Race-Car Demonstration for Policy Iteration



The smaller diagram above keeps the transition probabilities and rewards close to the computation. Let $\gamma = 0.9$. Suppose the initial policy is

$$\pi_0(\mathit{Cool}) = \mathit{Slow}, \qquad \pi_0(\mathit{Warm}) = \mathit{Slow}.$$

Under this policy, solving the two Bellman equations gives approximately

$$V_{\pi_0}(\mathit{Cool}) = 10, \qquad V_{\pi_0}(\mathit{Warm}) = 10. \tag{9}$$

Now apply one-step lookahead for improvement.

$$q_{\pi_0}(\mathit{Cool}, \mathit{Slow}) = 1 + 0.9(10) = 10, \tag{10}$$
$$q_{\pi_0}(\mathit{Cool}, \mathit{Fast}) = 0.5[2 + 0.9(10)] + 0.5[2 + 0.9(10)] = 11, \tag{11}$$
$$q_{\pi_0}(\mathit{Warm}, \mathit{Slow}) = 0.5[1 + 0.9(10)] + 0.5[1 + 0.9(10)] = 10, \tag{12}$$
$$q_{\pi_0}(\mathit{Warm}, \mathit{Fast}) = -10. \tag{13}$$

Thus the improved policy is

$$\pi_1(\mathit{Cool}) = \mathit{Fast}, \qquad \pi_1(\mathit{Warm}) = \mathit{Slow}. \tag{14}$$

Interpretation. Policy iteration separates two ideas: first understand the current policy, then improve it. This makes the logic clear, although full policy evaluation may be costly in large problems.

7. Value Iteration

Value iteration combines policy improvement with a shortened policy evaluation step. Instead of evaluating a fixed policy completely, it directly applies the Bellman optimality backup.

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_k(s') \right], \qquad s \in \mathcal{S}. \tag{15}$$

Value Iteration

Parameter: small threshold $\theta > 0$.

Initialize: $V(s)$ arbitrarily for all $s \in \mathcal{S}^+$, with $V(\text{terminal}) = 0$.

Loop:

1. Set $\Delta \leftarrow 0$.
2. For each $s \in \mathcal{S}$:

$$v \leftarrow V(s),$$
$$V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s') \right],$$
$$\Delta \leftarrow \max(\Delta, |v - V(s)|).$$

3. Stop when $\Delta < \theta$.

Output: a deterministic greedy policy

$$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s') \right].$$

Note. The max operator performs policy improvement inside each backup. It selects the action whose one-step lookahead gives the largest estimated return.

7.1. In-place and Not-in-place Updates

The value-iteration algorithm shown above uses a single array V. Therefore, it is an **in-place** algorithm: once a state's value is updated, the new value may be used immediately in later updates in the same sweep. A **not-in-place** or two-array version keeps an old array V_k and writes all new values into a separate array V_{k+1}. In that version, every state in the sweep is updated from the same old values.

Small value-iteration illustration. In the race-car example below, the displayed sweeps are written in the not-in-place style for easy verification. Thus Sweep 2 uses $V_1(\mathit{Cool}) = 2$ and $V_1(\mathit{Warm}) = 1$ for both state updates. This gives

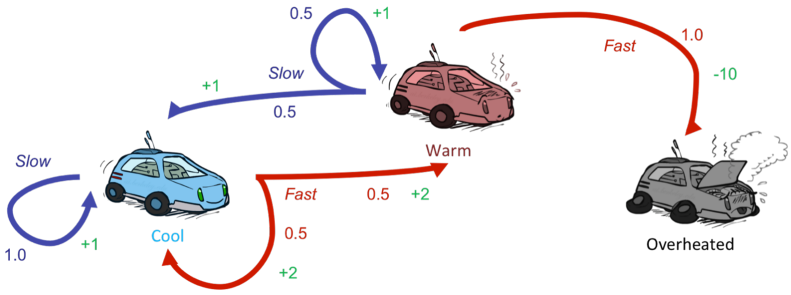
$$V_2(\mathit{Warm}) = 0.5[1 + 0.9(2)] + 0.5[1 + 0.9(1)] = 2.35.$$

If an in-place order updates Cool first and then Warm, the new value $V(\mathit{Cool}) = 3.35$ can already be used while updating Warm:

$$V(\mathit{Warm}) = 0.5[1 + 0.9(3.35)] + 0.5[1 + 0.9(1)] = 2.9575.$$

The final fixed point is the same under the usual convergence conditions, but intermediate numbers and speed of propagation can differ.

7.2. Race-Car Demonstration for Value Iteration



The smaller diagram above is used directly while reading the backups. Let $\gamma = 0.9$ and initialize $V_0(\mathit{Cool}) = V_0(\mathit{Warm}) = V_0(\mathit{Overheated}) = 0$. Applying the optimality backup gives:

Sweep 1. With all values initialized to zero,

$$V_1(\mathit{Cool}) = \max\{1 + 0.9(0),\ 2 + 0.9(0)\} = 2.00,$$
$$V_1(\mathit{Warm}) = \max\{1 + 0.9(0),\ -10 + 0.9(0)\} = 1.00.$$

The first sweep prefers Fast in Cool because of immediate reward +2, and Slow in Warm because Fast leads to overheating.

Sweep 2. Now use $V_1(\mathit{Cool}) = 2$ and $V_1(\mathit{Warm}) = 1$:

$$V_2(\mathit{Cool}) = \max\{1 + 0.9(2),\ 0.5[2 + 0.9(2)] + 0.5[2 + 0.9(1)]\} = 3.35,$$
$$V_2(\mathit{Warm}) = \max\{0.5[1 + 0.9(2)] + 0.5[1 + 0.9(1)],\ -10\} = 2.35.$$

The value increase is no longer only from immediate reward; it now includes discounted successor values.

Sweep 3. Using $V_2(\mathit{Cool}) = 3.35$ and $V_2(\mathit{Warm}) = 2.35$:

$$V_3(\mathit{Cool}) = \max\{4.015,\ 4.565\} = 4.565,$$
$$V_3(\mathit{Warm}) = \max\{3.565,\ -10\} = 3.565.$$

The greedy action in Cool is Fast, while the greedy action in Warm is Slow.

7.3. A 3 by 3 Grid Demonstration

Consider the following deterministic grid. The top-right cell is a good terminal reached with reward $+5$. The bottom-left cell is a bad terminal reached with reward -5. Other transitions have reward 0. Illegal moves leave the agent in the same cell. The update is read directly from the grid neighbourhood. Let $\gamma = 0.9$ and initialize all values to zero.

$(1,1)$	$(1,2)$	**G: +5**
$(2,1)$	$(2,2)$	$(2,3)$
B: -5	$(3,2)$	$(3,3)$

Value iteration produces the following sequence. All matrices are shown with the same display size.

$$k=0 \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \qquad k=1 \begin{bmatrix} 0 & 5 & 0 \\ 0 & 0 & 5 \\ 0 & 0 & 0 \end{bmatrix} \qquad k=2 \begin{bmatrix} 4.5 & 5 & 0 \\ 0 & 4.5 & 5 \\ 0 & 0 & 4.5 \end{bmatrix} \qquad k=3 \begin{bmatrix} 4.5 & 5 & 0 \\ 4.05 & 4.5 & 5 \\ 0 & 4.05 & 4.5 \end{bmatrix}$$

Highlighted computations for verification. Students should verify the coloured entries from the displayed matrices before proceeding.

- Green entries at $k = 1$: cells adjacent to the good terminal can move directly to it, so their value becomes 5.
- Blue entries at $k = 2$: verify cell $(2,2)$: the best successor has previous value 5, so $V_2(2,2) = 0 + 0.9(5) = 4.5$.
- Purple entries at $k = 3$: verify cell $(2,1)$: the best successor has previous value 4.5, so $V_3(2,1) = 0 + 0.9(4.5) = 4.05$.

This example shows how high value propagates backward from the good terminal. Negative terminal states matter if the agent is forced toward them; otherwise the max operator chooses actions that avoid them.

8. Asynchronous Dynamic Programming

Classical DP algorithms are often described as full sweeps over the state set. In **asynchronous dynamic programming**, the algorithm does not have to update every state in every sweep. Instead, it updates selected states in some order, using the most recent values available at the time of the update.

The basic asynchronous value-iteration backup for a selected state s is still

$$V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s') \right]. \tag{16}$$

Only the scheduling of updates changes. Rather than sweeping through all states equally, the algorithm may update one state, a small group of states, or states judged to be important.

Why this is useful. In a large grid, many states may be far from the agent or irrelevant to the current task. Instead of repeatedly updating all states, an asynchronous method can update states near the current trajectory, states whose successors recently changed, or states with large Bellman errors. This can focus computation where it is most useful.

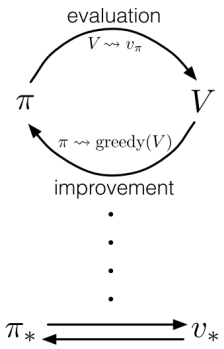
Convergence idea. For finite MDPs, asynchronous DP can still converge when updates are organized so that no relevant state is ignored forever. The practical message is that DP need not always be implemented as rigid full sweeps; it can be interleaved with interaction, simulation, or prioritized planning.

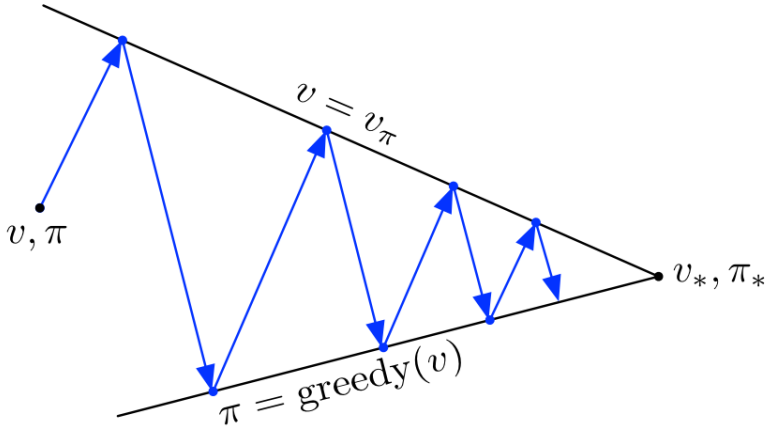
8.1. Where Asynchronous DP is Useful

- **Large tabular problems:** full sweeps may be costly, so selected-state updates reduce unnecessary computation.
- **Real-time planning:** updates can be concentrated around states the agent is currently visiting.
- **Prioritized updates:** states with large expected changes can be updated earlier.
- **Bridge to later methods:** many sample-based RL algorithms can be viewed as using partial, local, or sampled Bellman-style updates instead of complete expected sweeps.

9. Generalized Policy Iteration

Generalized policy iteration (GPI) is the general idea that policy evaluation and policy improvement interact with each other. Evaluation makes the value function more consistent with the current policy. Improvement makes the policy greedier with respect to the current value function. These two processes may be complete, partial, interleaved, or approximate.





Generalized policy iteration: evaluation improves V for the current policy; improvement makes π greedier with respect to the current V.

9.1. Algorithms as Instances of GPI

- **Policy iteration:** a coarse-grained form of GPI. The policy is evaluated almost completely, and then a separate greedy policy-improvement step is applied.
- **Value iteration:** a finer-grained form of GPI. Each optimality backup combines a short evaluation step with immediate improvement through the max operator.
- **Asynchronous DP:** a more flexible form. Evaluation/improvement-style backups are applied to selected states rather than to all states in uniform sweeps.
- **Monte Carlo and TD control methods:** sample-based GPI. Policy evaluation is learned from experience and policy improvement is performed gradually.

The important point is that evaluation and improvement need not be separated into large blocks. In practical RL, they often proceed together. As long as the value function becomes more accurate for the current policy and the policy becomes greedier with respect to the value function, the overall process follows the GPI logic.

10. Issues in Applying DP Methods

DP gives a clean solution route for finite MDPs, but its direct use depends on strong modelling and computational assumptions. The following issues explain where DP is useful, where it becomes expensive, and how the same ideas are carried forward in later reinforcement learning methods.

10.1. Full Sweeps over the State Space

Issue. Standard policy evaluation and value iteration normally apply backups to every state in repeated sweeps. This is clear and reliable for small tabular MDPs, but it becomes costly when the number of states is large.

How to handle it. For small problems, full sweeps are acceptable and help students verify every update. For larger problems, **asynchronous DP** can update selected states in a chosen order, provided all relevant states continue to be updated. In practice, states whose values are likely to change more can be prioritized.

10.2. Extracting a Policy from State Values

Issue. A state-value function $V(s)$ tells how good a state is, but it does not directly store which action should be taken. To extract a policy from V, the agent must perform a one-step lookahead using the model:

$$\pi(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma V(s')\Big].$$

How to handle it. In planning problems, this lookahead is natural because the model is known. If direct action selection is required without repeated lookahead, it can be useful to maintain action values $q(s, a)$ instead of only state values.

10.3. Policy May Stabilize before Values Fully Converge

Issue. In many control problems, the greedy policy can stop changing before the numerical values have fully converged. Continuing value sweeps may improve the decimal accuracy of V, but may not change the action selected in each state.

How to handle it. When the final objective is a good policy, monitor whether the greedy policy is changing, not only whether values have converged. This is the reason policy iteration separates policy evaluation from policy improvement, and why value iteration often stops when value changes are below a small threshold.

10.4. Need for a Known Model

Issue. DP backups require the environment dynamics, normally in the form $p(s', r \mid s, a)$ or equivalent transition and reward information. Without this model, the expected backup cannot be computed exactly.

How to handle it. Use DP when the model is available, such as in a simulator, puzzle, planning problem, or fully specified classroom MDP. When the model is not available, use model-free methods such as Monte Carlo or temporal-difference learning, which estimate values from sampled experience.

10.5. Finite and Discrete Action Sets

Issue. Classical tabular DP assumes that the action choices can be enumerated. This is convenient for grid worlds and small control examples, but not for continuous controls such as steering angle, throttle level, or robotic joint torque.

How to handle it. For small demonstrations, continuous actions may be discretized. For realistic continuous-action problems, later methods such as policy-gradient and actor-critic approaches are

more appropriate because they can represent policies over continuous action spaces.

10.6. Large or Continuous State Spaces

Issue. Tabular DP stores one value for each state. This becomes infeasible when the state space is very large or continuous. Even if storage is possible, many states may rarely be visited, making exhaustive updates inefficient.

How to handle it. Use DP mainly for finite, small, fully known problems. For larger problems, use approximation, abstraction, sampling, or value-function approximation. The important idea to carry forward is the **Bellman backup**: later methods approximate the same backup rather than computing it exactly for every state.

10.7. When DP is Appropriate

DP is most appropriate when the problem is a finite MDP, the model is known, and the state-action space is small enough for repeated backups. It is particularly useful for teaching, verifying Bellman equations, testing planning logic, and understanding the foundation of later RL algorithms.

DP is less appropriate when the model is unknown, the action space is continuous, or the number of states is too large for tabular storage and repeated sweeps. In such cases, the DP idea remains important, but the implementation shifts to sampling, approximation, or learning from direct interaction.

11. Review of Key Terms

Term	Short definition
Dynamic programming	A family of planning algorithms that use recursive value equations to compute policies and value functions.
Planning	Computing or improving a policy using a model rather than only direct interaction experience.
Bellman backup	An update that replaces a value estimate by an expected immediate reward plus discounted successor value.
Policy evaluation	Computing v_π for a fixed policy π.
Policy improvement	Constructing a better policy by acting greedily with respect to v_π or q_π.
Policy iteration	Alternating policy evaluation and policy improvement until the policy is stable.
Value iteration	Repeatedly applying the Bellman optimality backup to approach v_*.
Sweep	One pass through the states, applying an update to each state.
Model	The transition and reward information needed to compute expected backups.
In-place update	An update that writes the new value directly into the same array, so later updates may use the latest value.
Not-in-place update	A two-array update in which all new values are computed from the previous sweep's old values.
Asynchronous DP	A DP approach that updates states in a selected order rather than updating all states in every sweep.
Generalized policy iteration	The interaction of policy evaluation and policy improvement, possibly in complete, partial, interleaved, or approximate forms.

12. Review Questions

1. Define dynamic programming in the context of finite MDPs. Explain why classical DP is a planning method and state the information it requires from the environment model.
2. State the Bellman equation for $v_\pi(s)$. Explain how the equation separates immediate reward, discounting, successor-state values, and policy probabilities.
3. In iterative policy evaluation, why is the policy kept fixed? Explain what is updated during each sweep and why the stopping test uses Δ.
4. In the five-cell line example, states 1 and 5 are terminal with rewards -1 and $+1$ respectively, and all other transition rewards are 0. Starting from zero values and using $\gamma = 1$, compute the first two sweeps for the policy $\pi(L) = 0.8$, $\pi(R) = 0.2$ for states $2, 3, 4$.
5. For the same five-cell line, explain why the middle state has value 0 under the equal-probability policy but becomes positive under the mostly-right policy.
6. State the policy improvement theorem. Then explain, in words and using one equation, why choosing a greedy action with respect to v_π cannot make the policy worse in a finite MDP.
7. In the race-car problem, let $V(\textit{Cool}) = 2$, $V(\textit{Warm}) = 1$, $V(\textit{Overheated}) = 0$, and $\gamma = 0.9$. Compute the one-step lookahead values of Slow and Fast in the Cool state and identify the greedy action.
8. In the race-car problem, suppose $V(\textit{Cool}) = 3.35$ and $V(\textit{Warm}) = 2.35$. Compute the value-iteration backup for Warm and explain why the max operator avoids Fast.
9. In the race-car value-iteration example, compute the in-place update for Warm if Cool is updated first to 3.35 while Warm is still 1. Compare this with the not-in-place value 2.35 and explain why the two intermediate values differ.
10. A hospital has three treatment states: mild, serious, and recovered. Actions are standard treatment and aggressive treatment. Aggressive treatment may improve a serious patient faster but has a risk of side effects. Identify possible states, actions, rewards, and transition probabilities needed before DP can be applied.
11. A warehouse robot has a known simulator with 25 grid locations and four actions. It receives $+10$ for reaching the packing station, -5 for collision, and -1 per move. Formulate the Bellman optimality backup for one nonterminal location and explain how value iteration would update it.
12. In the 3 by 3 grid example, compute $V_2(2, 2)$ and $V_3(2, 1)$ from the displayed matrices. Clearly indicate which successor cell is chosen by the max operator.
13. Explain asynchronous dynamic programming. Under what condition on state updates can it still be expected to converge in finite tabular MDPs? Give one situation where asynchronous updating is more useful than full sweeps.
14. Explain generalized policy iteration. Show how policy iteration, value iteration, and a sample-based control method can each be viewed as different granularities of the same evaluation-improvement idea.
15. Compare policy iteration and value iteration in terms of policy evaluation effort, use of the max operator, stopping criterion, and practical behaviour when the policy stabilizes before the values fully converge.
16. A delivery robot has 100,000 possible map states and an unknown transition model. Explain why classical DP is difficult to apply directly. Suggest which later families of RL methods become relevant and why.

13. Points to Remember

- DP solves finite MDPs by converting Bellman equations into repeated update rules.

- Policy evaluation is prediction; policy improvement and value iteration address control.
- Policy iteration alternates between evaluating a policy and improving it.
- Value iteration directly applies the Bellman optimality backup.
- In-place and asynchronous updates can change the order and speed with which value information propagates.
- Generalized policy iteration explains how evaluation and improvement interact across many RL algorithms.
- DP is exact and elegant, but it assumes a known model and manageable finite state-action spaces.
- Understanding DP prepares the ground for TD learning, SARSA, Q-learning, and deep RL methods.

14. Required Reading

Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Required reading: Chapter 4, especially Sections 4.1 to 4.6, covering policy evaluation, policy improvement, policy iteration, value iteration, asynchronous dynamic programming, and generalized policy iteration. Section 4.7 may be read for efficiency considerations.

15. References

Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Cambridge, MA: MIT Press.

16. Acknowledgements

These lecture notes are based on and adapted from Chapter 4 of Sutton and Barto's *Reinforcement Learning: An Introduction*, Second Edition.

End of Topic -4